

Banking Management System

```
CREATE DATABASE BankingManagementSystem;
```

```
USE BankingManagementSystem;
```

```
mysql> USE BankingManagementSystem;  
Database changed
```

Creating tables

Customer Table

```
CREATE TABLE Customer (  
customer_id INT PRIMARY KEY  
name VARCHAR(100),  
phone VARCHAR(15),  
address VARCHAR(200)  
);
```

```
mysql> desc Customer;  
+-----+-----+-----+-----+-----+-----+  
| Field      | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| customer_id | int           | NO   | PRI | NULL    |       |  
| name        | varchar(100)  | YES  |     | NULL    |       |  
| phone       | varchar(15)   | YES  |     | NULL    |       |  
| address     | varchar(200)  | YES  |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
4 rows in set (0.01 sec)
```

Branch Table

```
CREATE TABLE Branch (  
branch_id INT PRIMARY KEY,  
branch_name VARCHAR(100),  
location VARCHAR(100)  
);
```

```
mysql> desc Branch;  
+-----+-----+-----+-----+-----+-----+  
| Field      | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| branch_id  | int           | NO   | PRI | NULL    |       |  
| branch_name | varchar(100)  | YES  |     | NULL    |       |  
| location   | varchar(100)  | YES  |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
3 rows in set (0.01 sec)
```

Account Table

```
CREATE TABLE Account (  
account_id INT PRIMARY KEY,  
customer_id INT,  
branch_id INT,  
account_type VARCHAR(50),  
balance DECIMAL(10,2),  
FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),  
FOREIGN KEY (branch_id) REFERENCES Branch(branch_id)  
);
```

```
mysql> desc Account;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| account_id     | int           | NO   | PRI | NULL    |       |
| customer_id    | int           | YES  | MUL | NULL    |       |
| branch_id      | int           | YES  | MUL | NULL    |       |
| account_type   | varchar(50)   | YES  |     | NULL    |       |
| balance        | decimal(10,2) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Transaction Table

```
CREATE TABLE Transaction (
transaction_id INT PRIMARY KEY,
account_id INT,
transaction_type VARCHAR(20),
amount DECIMAL(10,2),
transaction_date DATE,
FOREIGN KEY (account_id) REFERENCES Account(account_id)
);
```

```
mysql> desc Transaction;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| transaction_id | int           | NO   | PRI | NULL    |       |
| account_id     | int           | YES  | MUL | NULL    |       |
| transaction_type | varchar(20)   | YES  |     | NULL    |       |
| amount         | decimal(10,2) | YES  |     | NULL    |       |
| transaction_date | date          | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Loan Table

```

CREATE TABLE Loan (
loan_id INT PRIMARY KEY,
customer_id INT,
amount DECIMAL(10,2),
loan_type VARCHAR(50),
status VARCHAR(20),
FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);

```

```
mysql> desc Loan;
```

Field	Type	Null	Key	Default	Extra
loan_id	int	NO	PRI	NULL	
customer_id	int	YES	MUL	NULL	
amount	decimal(10,2)	YES		NULL	
loan_type	varchar(50)	YES		NULL	
status	varchar(20)	YES		NULL	

5 rows in set (0.01 sec)

Employee Table

```

CREATE TABLE Employee (
employee_id INT PRIMARY KEY,
name VARCHAR(100),
branch_id INT,
role VARCHAR(50),
FOREIGN KEY (branch_id) REFERENCES Branch(branch_id)
);

```

```
mysql> desc Employee;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| employee_id | int           | NO   | PRI | NULL    |       |
| name        | varchar(100)  | YES  |     | NULL    |       |
| branch_id   | int           | YES  | MUL | NULL    |       |
| role        | varchar(50)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

Insert Data into Tables

Customer Table

INSERT INTO Customer VALUES

(1,'Shraddha','9876543210','Udupi'),
 (2,'Amit','9876501234','Mangalore'),
 (3,'Riya','9988776655','Bangalore'),
 (4,'Kiran','9123456780','Manipal');

```
mysql> select * from Customer;
+-----+-----+-----+-----+
| customer_id | name      | phone      | address |
+-----+-----+-----+-----+
| 1           | Shraddha  | 9876543210 | Udupi   |
| 2           | Amit      | 9876501234 | Mangalore |
| 3           | Riya      | 9988776655 | Bangalore |
| 4           | Kiran     | 9123456780 | Manipal |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

Branch Table

INSERT INTO Branch VALUES

(101,'Main Branch','Udupi'),
 (102,'City Branch','Mangalore');

```
mysql> select * from Branch;
```

branch_id	branch_name	location
101	Main Branch	Udupi
102	City Branch	Mangalore

```
2 rows in set (0.00 sec)
```

Account Table

INSERT INTO Account VALUES

(1001,1,101,'Savings',60000),

(1002,2,102,'Current',40000),

(1003,3,101,'Savings',80000);

```
mysql> select * from Account;
```

account_id	customer_id	branch_id	account_type	balance
1001	1	101	Savings	60000.00
1002	2	102	Current	40000.00
1003	3	101	Savings	80000.00

```
3 rows in set (0.00 sec)
```

Transaction Table

INSERT INTO Transaction VALUES

(1,1001,'Deposit',10000,'2026-05-01'),

(2,1001,'Withdrawal',5000,'2026-05-05'),

(3,1003,'Deposit',20000,'2026-05-08');

```
mysql> select * from Transaction;
```

transaction_id	account_id	transaction_type	amount	transaction_date
1	1001	Deposit	10000.00	2026-05-01
2	1001	Withdrawal	5000.00	2026-05-05
3	1003	Deposit	20000.00	2026-05-08

```
3 rows in set (0.00 sec)
```

Loan Table

INSERT INTO Loan VALUES

(201,1,500000,'Home Loan','Approved'),

(202,2,100000,'Education Loan','Pending');

```
mysql> select * from Loan;
```

loan_id	customer_id	amount	loan_type	status
201	1	500000.00	Home Loan	Approved
202	2	100000.00	Education Loan	Pending

```
2 rows in set (0.00 sec)
```

Employee Table

INSERT INTO Employee VALUES

(301,'Rahul',101,'Manager'),

(302,'Sneha',102,'Cashier');

```
mysql> select * from Employee;
```

employee_id	name	branch_id	role
301	Rahul	101	Manager
302	Sneha	102	Cashier

```
2 rows in set (0.00 sec)
```

Queries

1. Retrieve all customers (Basic SELECT)

SELECT * FROM Customer;

```
mysql> -- 1. Retrieve all customers (Basic SELECT)
mysql> SELECT * FROM Customer;
+-----+-----+-----+-----+
| customer_id | name   | phone   | address |
+-----+-----+-----+-----+
| 1           | Shraddha | 9876543210 | Udupi   |
| 2           | Amit    | 9876501234 | Mangalore |
| 3           | Riya    | 9988776655 | Bangalore |
| 4           | Kiran   | 9123456780 | Manipal |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

2. Display accounts with balance greater than 50000 (WHERE clause)

SELECT * FROM Account WHERE balance > 50000;

```
mysql> -- 2. Display accounts with balance greater than 50000 (WHERE clause)
mysql> SELECT * FROM Account
    -> WHERE balance > 50000;
+-----+-----+-----+-----+-----+
| account_id | customer_id | branch_id | account_type | balance |
+-----+-----+-----+-----+-----+
| 1001       | 1           | 101       | Savings      | 60000.00 |
| 1003       | 3           | 101       | Savings      | 80000.00 |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

3. Display transaction details with account information (2-table INNER JOIN)

```
SELECT t.transaction_id,
       t.transaction_type,
       t.amount,
       a.account_id,
       a.account_type
```

FROM Transaction t

INNER JOIN Account a

ON t.account_id = a.account_id;

transaction_id	transaction_type	amount	account_id	account_type
1	Deposit	10000.00	1001	Savings
2	Withdrawal	5000.00	1001	Savings
3	Deposit	20000.00	1003	Savings

3 rows in set (0.00 sec)

4. Display customer, account, and branch details (3-table JOIN)

SELECT c.name,

a.account_id,

b.branch_name,

b.location

FROM Customer c

JOIN Account a

ON c.customer_id = a.customer_id

JOIN Branch b

ON a.branch_id = b.branch_id;

name	account_id	branch_name	location
Shraddha	1001	Main Branch	Udupi
Riya	1003	Main Branch	Udupi
Amit	1002	City Branch	Mangalore

3 rows in set (0.00 sec)

5. Count number of accounts per branch (GROUP BY)

SELECT branch_id,

```
        COUNT(account_id) AS total_accounts
FROM Account
GROUP BY branch_id;
```

```
+-----+-----+
| branch_id | total_accounts |
+-----+-----+
|         101 |                2 |
|         102 |                1 |
+-----+-----+
2 rows in set (0.00 sec)
```

6. Display branches having total deposits greater than 1,00,000 (HAVING)

```
SELECT branch_id,
        SUM(balance) AS total_deposit
FROM Account
GROUP BY branch_id
HAVING SUM(balance) > 100000;
```

```
+-----+-----+
| branch_id | total_deposit |
+-----+-----+
|         101 | 140000.00 |
+-----+-----+
1 row in set (0.00 sec)
```

7. Retrieve customers whose account balance is greater than the average account balance (Subquery)

```
SELECT c.name,
        a.balance
FROM Customer c
JOIN Account a
ON c.customer_id = a.customer_id
```

WHERE a.balance >

(

SELECT AVG(balance)

FROM Account

);

```
+-----+-----+
| name | balance |
+-----+-----+
| Riya | 80000.00 |
+-----+-----+
1 row in set (0.00 sec)
```

8. Retrieve customers whose total transaction amount exceeds that of a specific customer (Correlated Subquery)

SELECT c.name,

SUM(t.amount) AS total_transaction

FROM Customer c

JOIN Account a

ON c.customer_id = a.customer_id

JOIN Transaction t

ON a.account_id = t.account_id

GROUP BY c.customer_id, c.name

HAVING SUM(t.amount) >

(

SELECT SUM(t2.amount)

FROM Customer c2

JOIN Account a2

ON c2.customer_id = a2.customer_id

JOIN Transaction t2

```
ON a2.account_id = t2.account_id
WHERE c2.customer_id = 1
);
```

```
+-----+-----+
| name | total_transaction |
+-----+-----+
| Riya |          20000.00 |
+-----+-----+
1 row in set (0.00 sec)
```

9. Display all customers including those without accounts (LEFT JOIN)

```
SELECT c.name,
       a.account_id
FROM Customer c
LEFT JOIN Account a
ON c.customer_id = a.customer_id;
```

```
+-----+-----+
| name   | account_id |
+-----+-----+
| Shraddha |        1001 |
| Amit    |        1002 |
| Riya    |        1003 |
| Kiran   |         NULL |
+-----+-----+
4 rows in set (0.00 sec)
```

10. Retrieve accounts with no transactions (NOT EXISTS)

```
SELECT *
FROM Account a
WHERE NOT EXISTS
(
  SELECT *
  FROM Transaction t
```

```
WHERE t.account_id = a.account_id  
);
```

account_id	customer_id	branch_id	account_type	balance
1002	2	102	Current	40000.00

1 row in set (0.00 sec)

Stored Procedures

1. Procedure to transfer funds between two accounts

```
DELIMITER //
```

```
CREATE PROCEDURE TransferFunds(  
    IN fromAcc INT,  
    IN toAcc INT,  
    IN amt DECIMAL(10,2)  
)  
BEGIN
```

```
    UPDATE Account  
    SET balance = balance - amt  
    WHERE account_id = fromAcc;
```

```
    UPDATE Account  
    SET balance = balance + amt  
    WHERE account_id = toAcc;
```

END //

DELIMITER ;

Execute Procedure

CALL TransferFunds(1001,1002,5000);

```
mysql> SELECT * FROM Account;
```

account_id	customer_id	branch_id	account_type	balance
1001	1	101	Savings	55000.00
1002	2	102	Current	45000.00
1003	3	101	Savings	80000.00

```
3 rows in set (0.00 sec)
```

2. Procedure to calculate monthly interest for savings accounts

CREATE PROCEDURE CalculateInterest()

BEGIN

UPDATE Account

SET balance = balance + (balance * 0.02)

WHERE account_type = 'Savings';

END //

DELIMITER ;

Execute Procedure

CALL CalculateInterest();

```
mysql> SELECT * FROM Account;
```

account_id	customer_id	branch_id	account_type	balance
1001	1	101	Savings	56100.00
1002	2	102	Current	45000.00
1003	3	101	Savings	81600.00

```
3 rows in set (0.00 sec)
```

Trigger Questions

1. Trigger to prevent withdrawal if balance falls below minimum balance

```
CREATE TRIGGER MinimumBalanceCheck
```

```
BEFORE INSERT ON Transaction
```

```
FOR EACH ROW BEGIN
```

```
DECLARE current_balance DECIMAL(10,2);
```

```
SELECT balance INTO current_balance
```

```
FROM Account
```

```
WHERE account_id = NEW.account_id;
```

```
IF NEW.transaction_type = 'Withdrawal'
```

```
AND (current_balance - NEW.amount) < 1000 THEN
```

```
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT = 'Minimum balance should be maintained';
```

```
END IF;
```

```
END //
```

```
DELIMITER ;
```

INSERT INTO Transaction

VALUES (4,1002,'Withdrawal',39500,'2026-06-11');

```
mysql> select * from transaction;
```

transaction_id	account_id	transaction_type	amount	transaction_date
1	1001	Deposit	10000.00	2026-05-01
2	1001	Withdrawal	5000.00	2026-05-05
3	1003	Deposit	20000.00	2026-05-08
4	1002	Withdrawal	39500.00	2026-06-11

4 rows in set (0.03 sec)

2. Trigger to automatically update account balance after transaction insertion

DELIMITER //

CREATE TRIGGER UpdateBalanceAfterTransaction
AFTER INSERT ON Transaction
FOR EACH ROW
BEGIN

IF NEW.transaction_type = 'Deposit' THEN

UPDATE Account
SET balance = balance + NEW.amount
WHERE account_id = NEW.account_id;

ELSEIF NEW.transaction_type = 'Withdrawal' THEN

UPDATE Account
SET balance = balance - NEW.amount
WHERE account_id = NEW.account_id;

END IF;

END //

DELIMITER ;

```
mysql> SELECT * FROM Account
      -> WHERE account_id = 1001;
+-----+-----+-----+-----+-----+
| account_id | customer_id | branch_id | account_type | balance |
+-----+-----+-----+-----+-----+
|      1001 |           1 |        101 | Savings      | 61100.00 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```